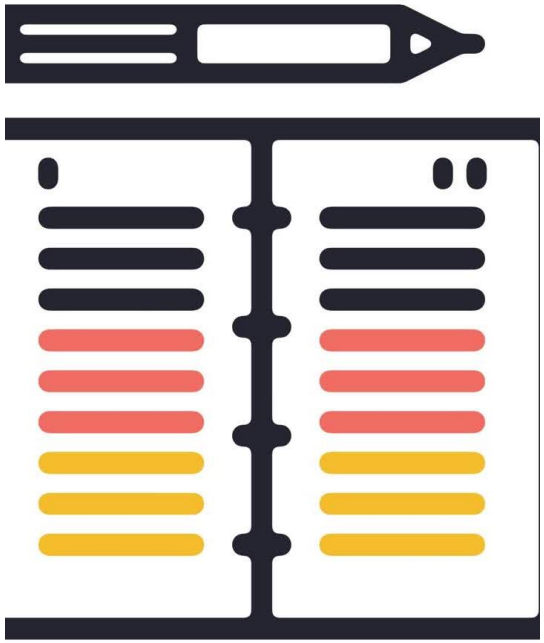
# Mastering Requirements Engineering

- From Elicitation to Validation



# What You'll Learn Today

- 📋 Define Requirements Engineering (RE)
- 🔍 Explain the RE process
- ✏️ Apply techniques like user stories and SRS
- 🚧 Identify common challenges

# What’s the Most Critical Phase in Software Development?

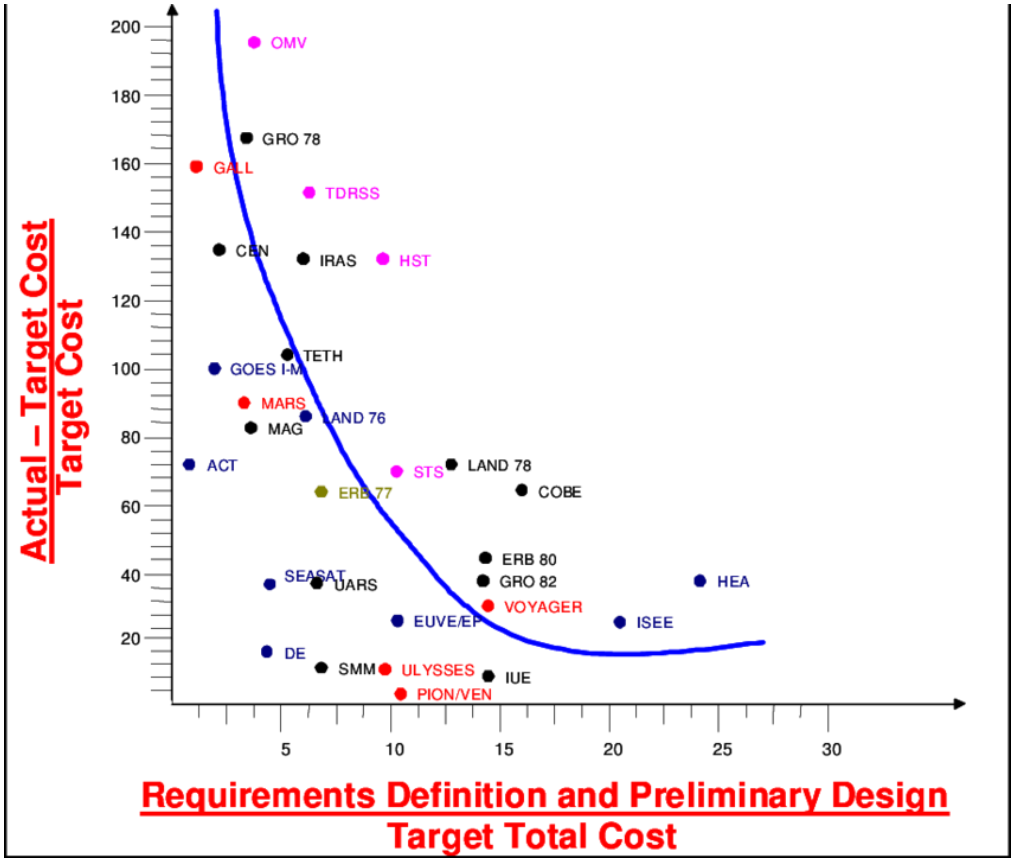
- 1. Coding
- 2. Testing
- 3. Requirements Engineering
- 4. Design

# Defining Requirements Engineering

- According to IEEE, requirement engineering is "the process through which developers and users discover, review, articulate, and understand the users' needs and the constraints on the software and development.

# Why RE Matters

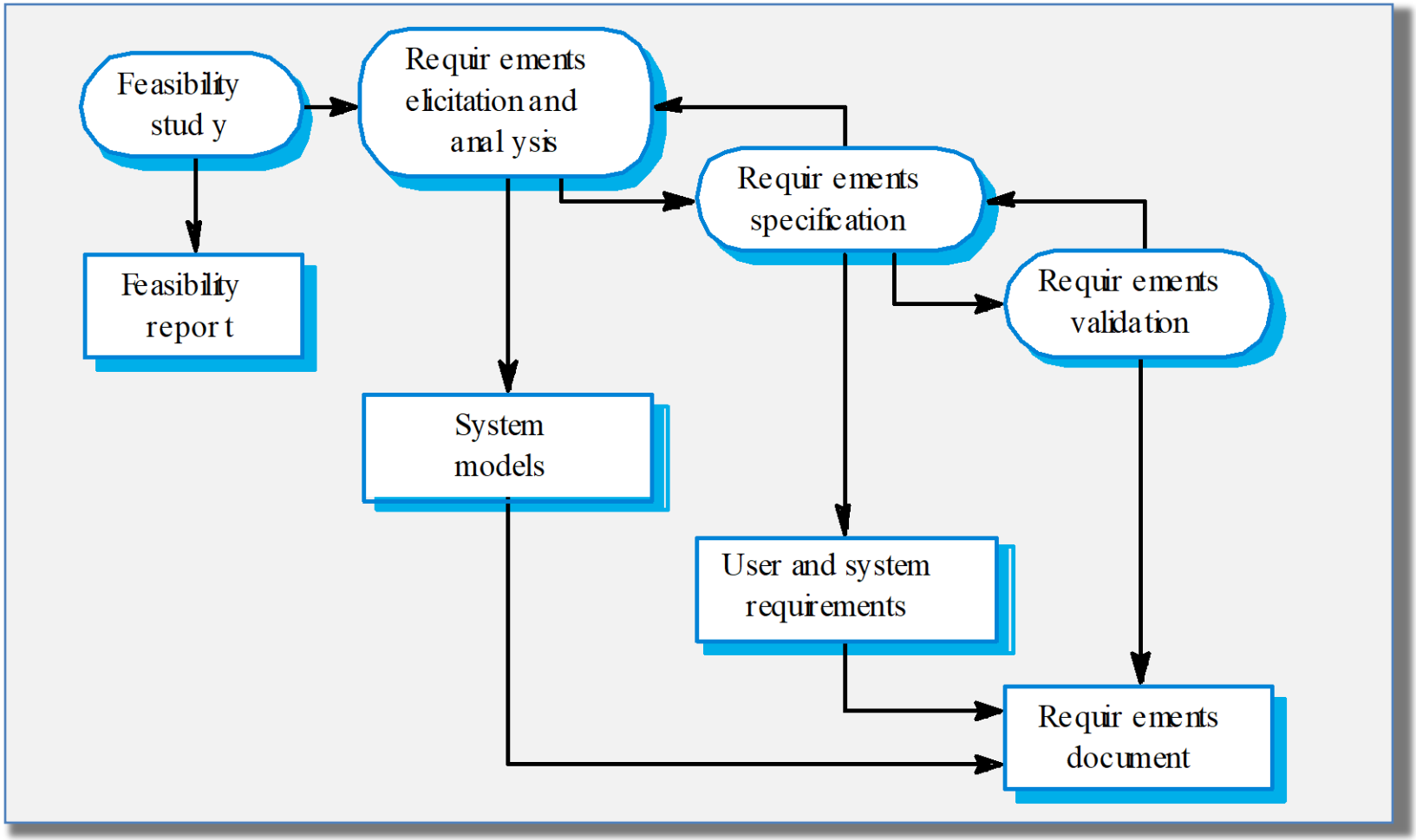
- Fixing a post-release error costs 100x more! (Source: IBM)
- Graph: Cost escalation



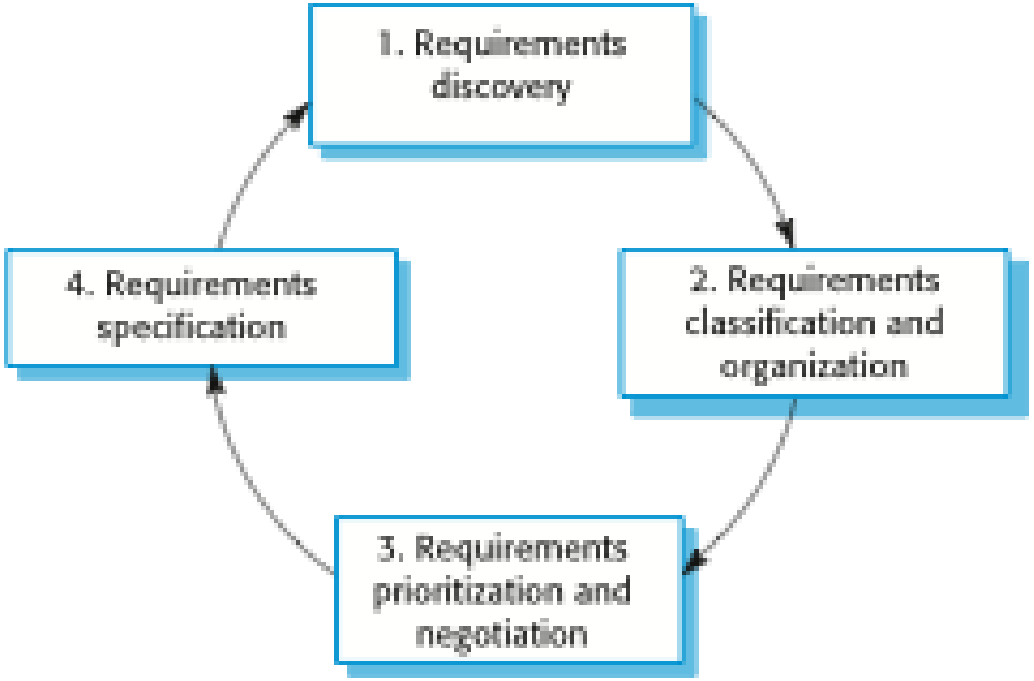
# Requirements Engineering Processes

- 1. Requirements elicitation;
  - What services do the end-users require of the system?
- 2. Requirements analysis;
  - How do we classify, prioritise and negotiate requirements?
- 3. Requirements Specification;
- Formal documentation of requirements in a structured format.
- 4. Requirements validation;
  - Does the proposed system do what the users require?
- 5. Requirements management.
  - How do we manage the (sometimes inevitable) changes to the requirements document?

# The Requirements Engineering Process



# The requirements elicitation and analysis process



# Example

**Patient records system**

**(Elicitation)** 1. Talk to patients, doctors, nurses, receptionists, managers to find out

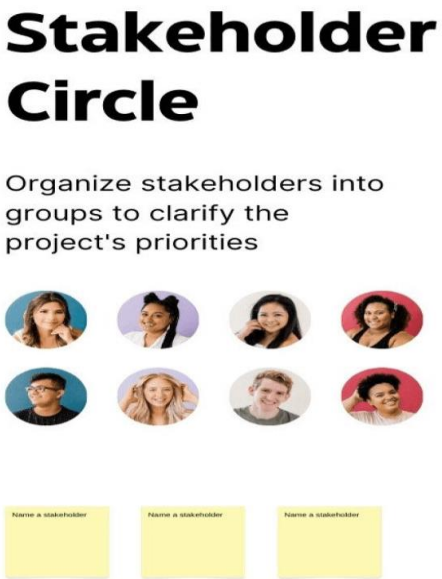
Current system practise, legal restrictions DPA, problems with current system, needs for improvement, security issues, costs

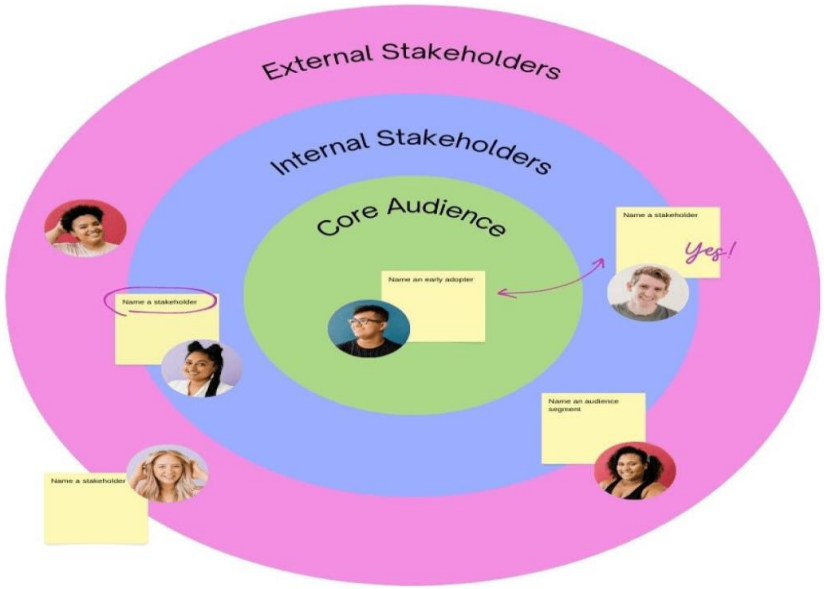
**(Elicitation)** 2.Develop draft documentation and review what is most important, what will it cost, what is the timescale, is new hardware required

**(Validation)** 3. Send requirements to end users. Present them with Q&A. Go back to step 1, discuss requirements again

**(Management)** 4.Have a yearly review of requirements between all stakeholders. Have a system of reviewing the cost and feasability of change to system

# Stakeholders



# Stakeholders in the MHC-PMS

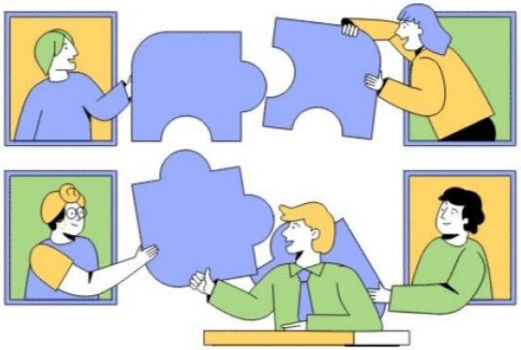
- Patients whose information is recorded in the system.
- Doctors who are responsible for assessing and treating patients.
- Nurses who coordinate the consultations with doctors and administer some treatments.
- Medical receptionists who manage patients' appointments.
- IT staff who are responsible for installing and maintaining the system.

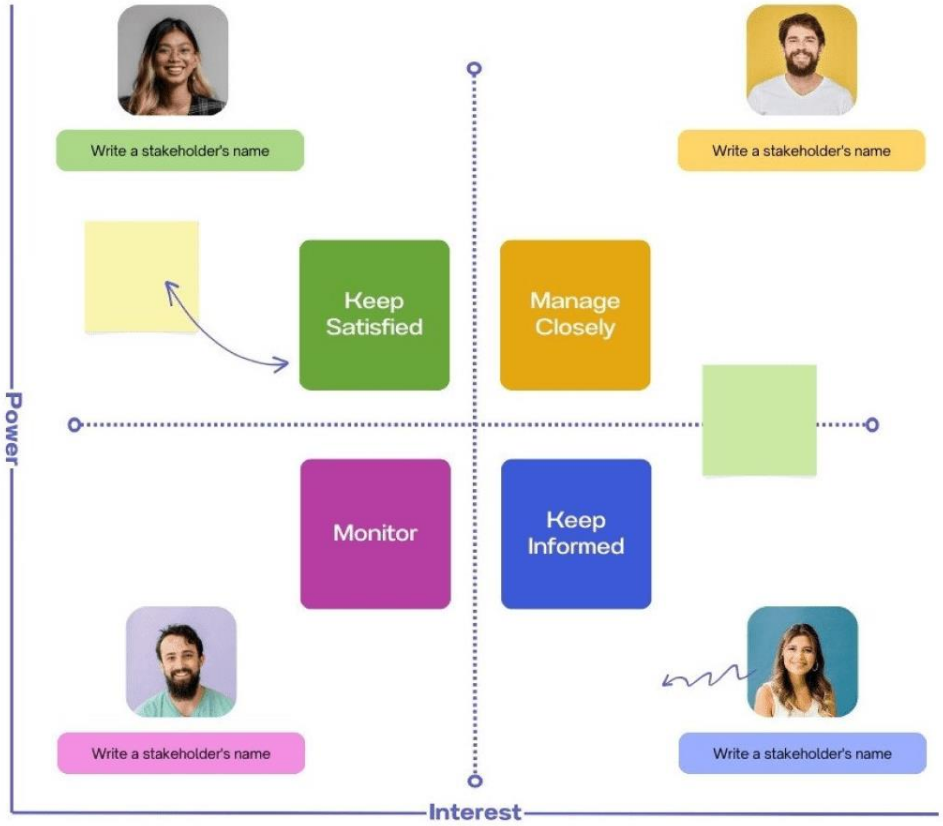
# Stakeholders in the MHC-PMS

- A medical ethics manager who must ensure that the system meets current ethical guidelines for patient care.
- Health care managers who obtain management information from the system.
- Medical records staff who are responsible for ensuring that system information can be maintained and preserved, and that record keeping procedures have been properly implemented.

# Stakeholder Map

Let's organize stakeholders according the power-interest matrix so we can figure out how to approach them better





# Comparison: Business vs. User vs. System Requirements

| Aspect | Business Requirements (What the Business Needs) | User Requirements (What Users Expect to Do) | System Requirements (How the System Will Work) |
|---|---|---|---|
| **Focus** | Business objectives and goals | Tasks and actions users must be able to perform | Technical specifications and implementation |
| **Purpose** | Defines the reason for the system or project | Defines user expectations and interactions | Defines how the system will meet business and user needs |
| **Scope** | High-level, strategic | Functional needs from the user's perspective | Detailed, technical |
| **Stakeholders** | Business executives, regulatory bodies, management | Students, instructors, administrators | Developers, IT teams, system architects |
| **Example (Energy Utility - NamPower Automation Project)** | "Improve power outage response time by 30%" | "Customers should receive real-time outage notifications" | "The system should send real-time outage alerts via SMS and email" |
| **Example (E-Commerce Website)** | "Expand to five new countries for international sales" | "Users should be able to pay in different currencies" | "The system must support multi-currency transactions and integrate with international payment gateways" |
| **Example (Mobile Banking App)** | "Increase customer engagement by 20%" | "Users should be able to check account balance and transfer funds via mobile" | "The system must provide secure authentication and support instant transactions" |
| **Example (E-Learning Platform)** | "Provide accessible and scalable online education" | "Students should be able to enroll, attend virtual classes, and complete assignments" | "The system must support video streaming, quizzes, and progress tracking for stu |

# User and system requirements reminder

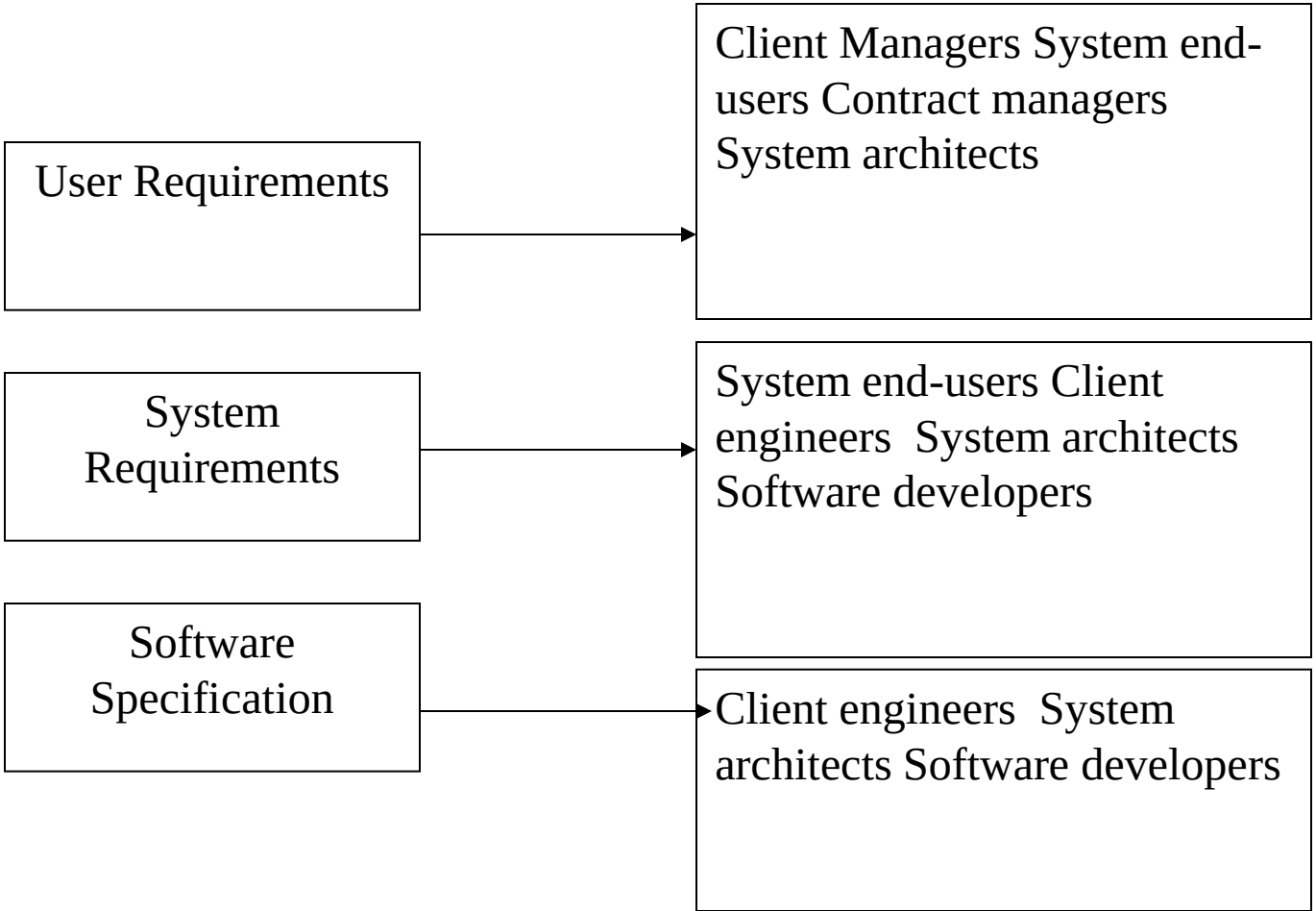
**User requirement definition**

1. The MHC-PMS shall generate monthly management reports showing the cost of drugs prescribed by each clinic during that month.

**System requirements specification**

**1.1** On the last working day of each month, a summary of the drugs prescribed, their cost and the prescribing clinics shall be generated.
**1.2** The system shall automatically generate the report for printing after 17.30 on the last working day of the month.
**1.3** A report shall be created for each clinic and shall list the individual drug names, the total number of prescriptions, the number of doses prescribed and the total cost of the prescribed drugs.
**1.4** If drugs are available in different dose units (e.g. 10mg, 20 mg, etc.) separate reports shall be created for each dose unit.
**1.5** Access to all cost reports shall be restricted to authorized users listed on a management access control list.

# Requirements Targets



# Requirements Types:

1. Functional requirements: services the system should provide
2. Non-functional requirements: constraints on the services of functions offered by the system. e.g. speed, time to market
3. Domain requirements: related to the application domain of the system (may be functional or non-functional requirements)

# Functional requirements

- Functionality or services that the system is expected to provide.
- Functional requirements may also explicitly state what the system shouldn't do.
- Functional requirements specification should be:
  - Complete: All services required by the user should be defined
  - Consistent: should not have contradictory definition (also avoid ambiguity→ don't leave room for different interpretations)

# Non-Functional requirements

- Requirements that are not directly concerned with the specific functions delivered by the system
- Typically relate to the system as a whole rather than the individual system features
- Often could be deciding factor on the survival of the system (e.g. reliability, cost, response time)

# Non-Functional requirements classifications:

**Non-Functional Requirements**

Product requirements

- Efficiency
- Reliability
- Portability
- Usability
- Performance
- Space

Organizational requirements

- Delivery
- Implementation
- Standards

External requirements

- Interoperability
- Ethics
- Legislative
- Privacy
- Safety

# Examples of nonfunctional requirements in the MHC-PMS

**Product requirement**
The MHC-PMS shall be available to all clinics during normal working hours (Mon–Fri, 0830–17.30). Downtime within normal working hours shall not exceed five seconds in any one day.

**Organizational requirement**
Users of the MHC-PMS system shall authenticate themselves using their health authority identity card.

**External requirement**
The system shall implement patient privacy provisions as set out in HStan-03-2006-priv.

# How to Gather Requirements

- Interviews
- Surveys
- Workshops
- Prototyping
- Scenario
- Ethnography
- JAD

# SMART Requirements

Specific | Measurable | Achievable | Relevant | Time-bound

❌ 'The app should be fast.'

✅ 'Process payments in <3 seconds with 10k users.'

# Fix These Requirements!

Vague: 'Users want easy navigation for the e-learning platform.'

Group Task: Rewrite using SMART criteria

# Pitfalls in Requirements Engineering

- 🚧 Changing Requirements
- 🗣️ Communication Gaps

# Requirements checking with User

- Validity. Do requirements match customer's real needs?
- Consistency. Are there any requirements conflicts?
- Completeness. Are all functions required by the customer included?
- Realism. Can the requirements be implemented given available budget and technology
- Verifiability. Can the requirements be checked?
- How to check
  - *Requirements reviews*
  - *Prototyping*
  - *Test-case generation*

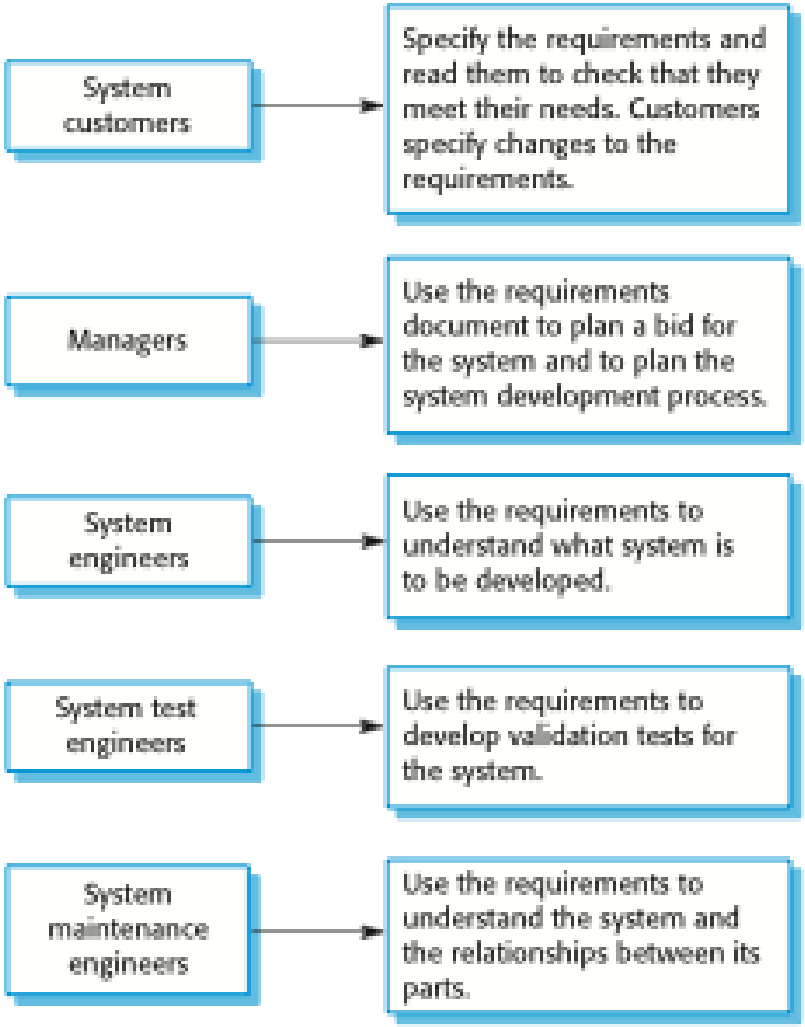
# The software requirements document

The software requirements document is the official statement of what is required of the system developers.

Should include both a definition of user requirements and a specification of the system requirements.

It is NOT a design document. As far as possible, it should set of WHAT the system should do rather than HOW it should do it.

# Users of a requirements document



# Introduction to User Stories

**What are User Stories?**

A short, simple description of a feature **from the perspective of the user**.

Used in **Agile development** to ensure user needs are clearly defined.

**User Story Format:**
*"As a [user role], I want [feature] so that [benefit]."*

*"Let's look at an example."*

# Writing Effective User Stories

✓ **Bad User Story:**
*"I want to reset my password."*

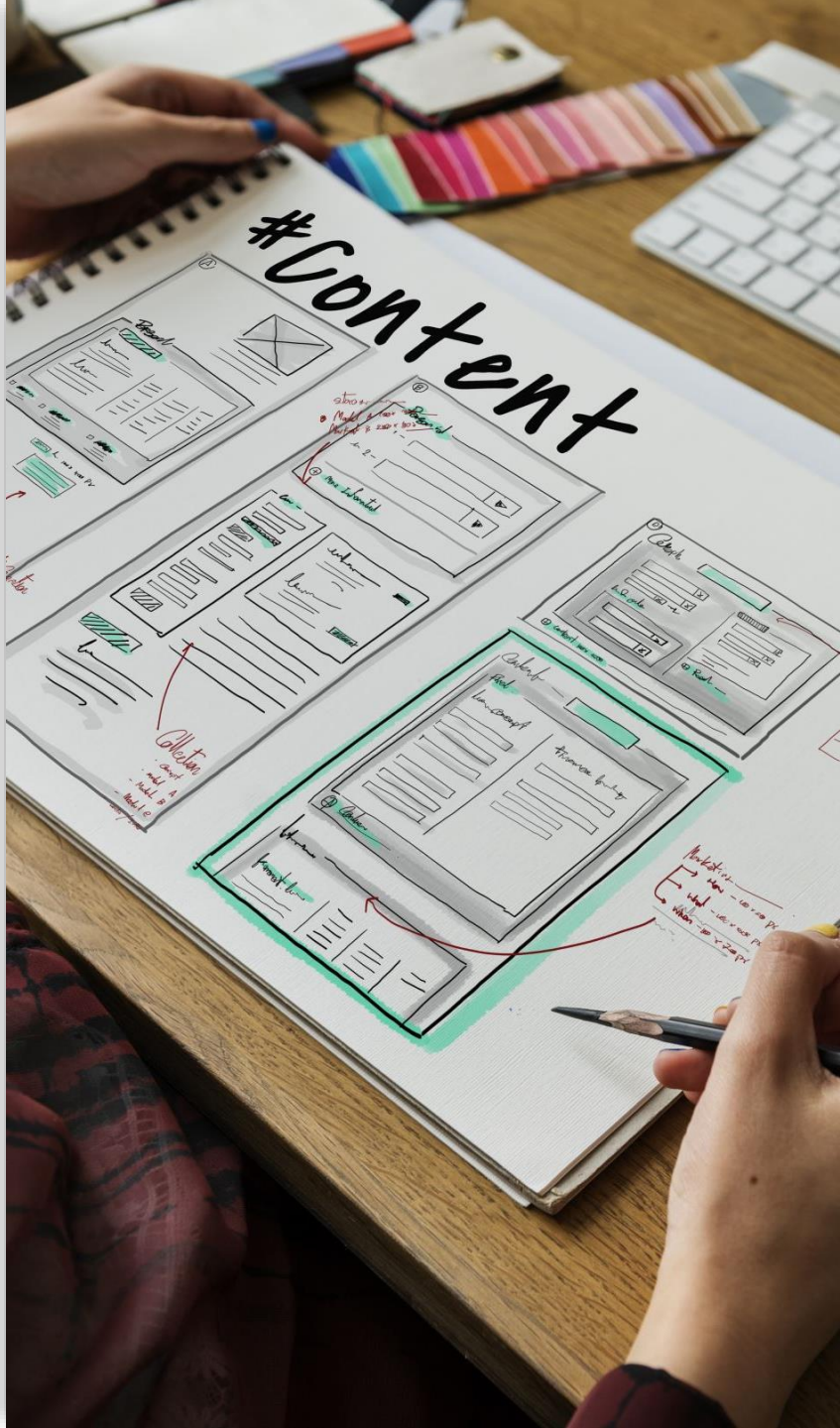
✓ **Better User Story (Specific & Actionable):**
*"As a user, I want to reset my password via email verification so that I can regain access securely."*

*"Notice how this version clearly defines WHO (the user), WHAT (reset password via email), and WHY (regain access securely)."*

# Group Exercise – Create Your Own User Stories

**Activity:**

- Break into groups and select **one feature** from your project.
- Write **three user stories** following the format:
  *As a [user role], I want [feature] so that [benefit].*
- Discuss how they can be **refined and improved**.



# Each Sprint should have clearly defined tasks from the Product Backlog.

| User Story | Task | Assignee | Priority | Status |
|---|---|---|---|---|
| "As a student, I want to log in to my account so that I can access my lessons." | Develop login UI | Front-End Dev | High | In Progress |
| | Implement authentication API | Back-End Dev | High | To Do |
| "As a teacher, I want to assign lessons to students so that I can manage student learning." | Create teacher dashboard | Front-End Dev | Medium | To Do |
| "As a student, I want AI to recommend lessons based on my progress so that I can improve my learning." | Develop AI recommendation model | AI/ML Specialist | High | In Progress |
| | Train AI model using dataset | AI/ML Specialist | High | In Progress |
| "As an admin, I want to manage users so that I can ensure proper platform use. | Develop - Ability to deactivate/reactivate accounts<br>- Role-based access control | Back-End Dev | High | To Do |

# Validating & Prioritising User Stories

- ✓ **Use MoSCoW Prioritisation:**
- **Must-Have:** Critical features (e.g., login functionality).
- **Should-Have:** Important, but not essential (e.g., dark mode).
- **Could-Have:** Nice-to-have features (e.g., animated UI elements).
- **Won't-Have:** Features that are **out of scope** for now.

# The structure of a requirements document

| Chapter | Description |
|---|---|
| Preface | This should define the expected readership of the document and describe its version history, including a rationale for the creation of a new version and a summary of the changes made in each version. |
| Introduction | This should describe the need for the system. It should briefly describe the system's functions and explain how it will work with other systems. It should also describe how the system fits into the overall business or strategic objectives of the organization commissioning the software. |
| Glossary | This should define the technical terms used in the document. You should not make assumptions about the experience or expertise of the reader. |
| User requirements definition | Here, you describe the services provided for the user. The nonfunctional system requirements should also be described in this section. This description may use natural language, diagrams, or other notations that are understandable to customers. Product and process standards that must be followed should be specified. |
| System architecture | This chapter should present a high-level overview of the anticipated system architecture, showing the distribution of functions across system modules. Architectural components that are reused should be highlighted. |

# The structure of a requirements document

| Chapter | Description |
|---|---|
| System requirements specification | This should describe the functional and nonfunctional requirements in more detail. If necessary, further detail may also be added to the nonfunctional requirements. Interfaces to other systems may be defined. |
| System models | This might include graphical system models showing the relationships between the system components and the system and its environment. Examples of possible models are object models, data-flow models, or semantic data models. |
| System evolution | This should describe the fundamental assumptions on which the system is based, and any anticipated changes due to hardware evolution, changing user needs, and so on. This section is useful for system designers as it may help them avoid design decisions that would constrain likely future changes to the system. |
| Appendices | These should provide detailed, specific information that is related to the application being developed; for example, hardware and database descriptions. Hardware requirements define the minimal and optimal configurations for the system. Database requirements define the logical organization of the data used by the system and the relationships between data. |
| Index | Several indexes to the document may be included. As well as a normal alphabetic index, there may be an index of diagrams, an index of functions, and so on. |

# Key points

- The software requirements document is an agreed statement of the system requirements. It should be organized so that both system customers and software developers can use it.
- The requirements engineering process is an iterative process including requirements elicitation, specification and validation.
- Requirements elicitation and analysis is an iterative process that can be represented as a spiral of activities – requirements discovery, requirements classification and organization, requirements negotiation and requirements documentation.

# Keep Learning!

- 📚 Read: IEEE Standards for SRS
- 🛠️ Draft an SRS for a your project
- 📎 Tools: Figma, Jira, Trello

# One Takeaway, One Question

- 1. What’s your key takeaway?
- 2. What’s still unclear?